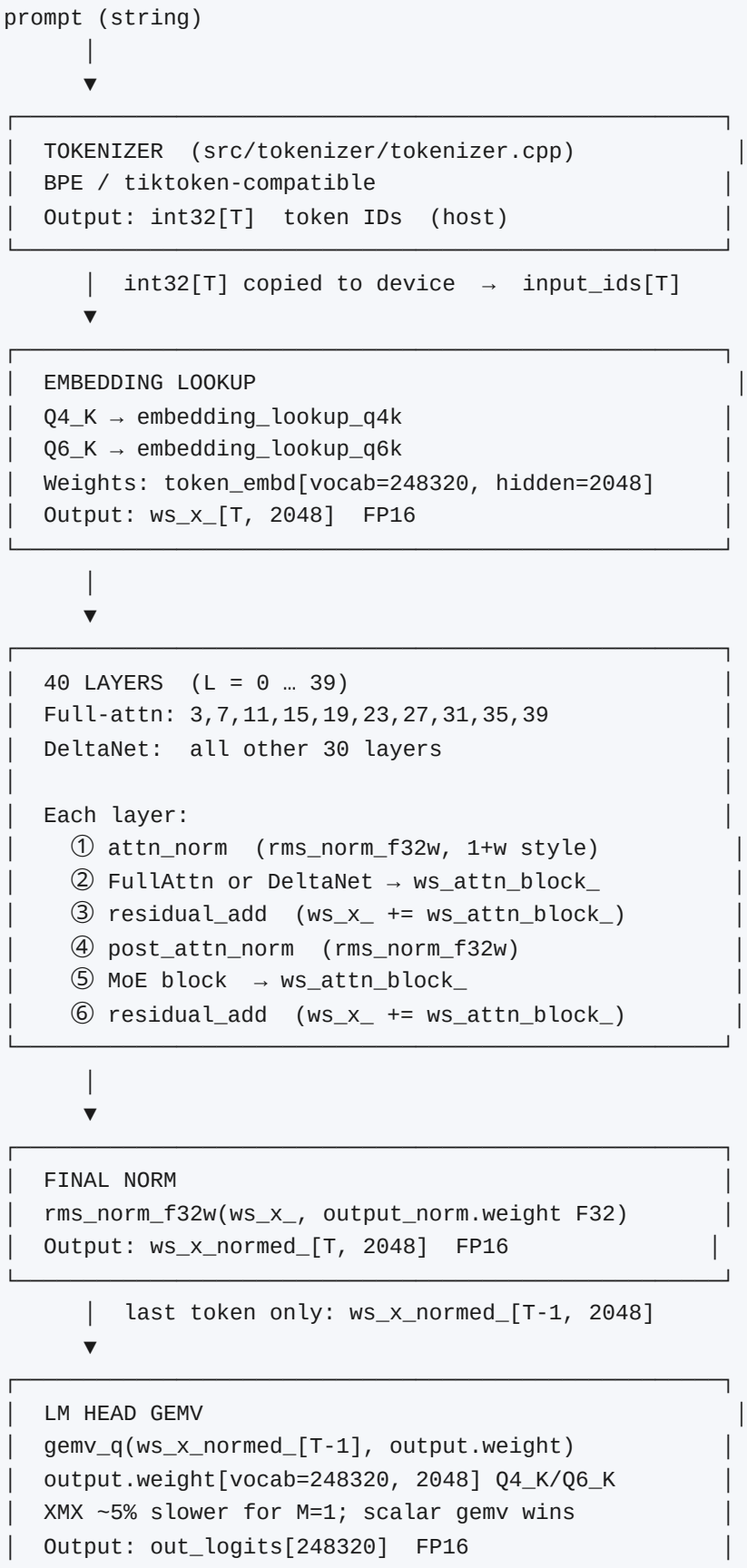
IE Inference Engine — Complete Architecture Diagrams

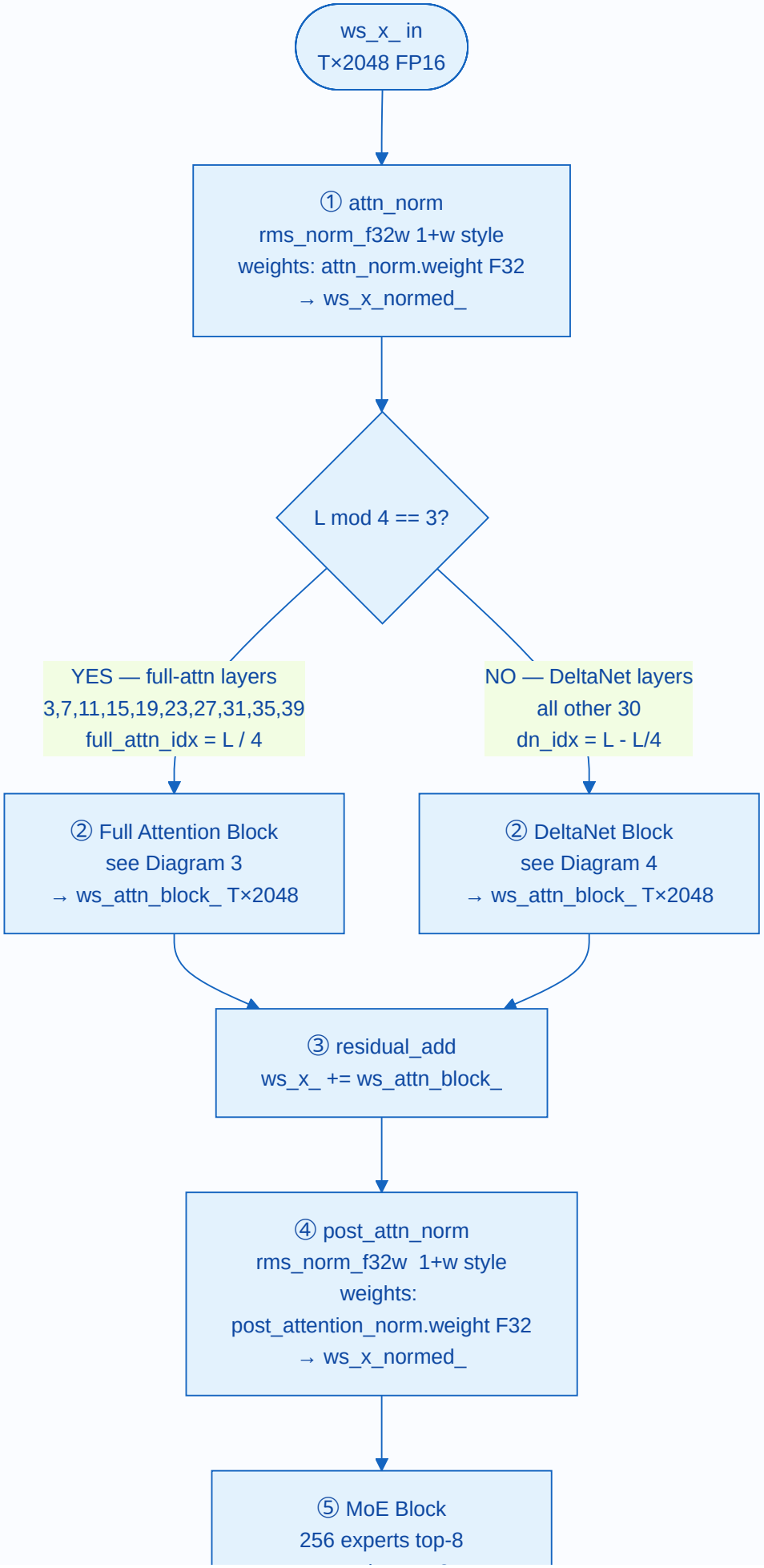
Model: Qwen3.6-35B-A3B | Hardware: Intel Arc Pro B70 (BMG-G31 C0) | Runtime: SYCL / Level Zero | Source verified: 2026-05-01

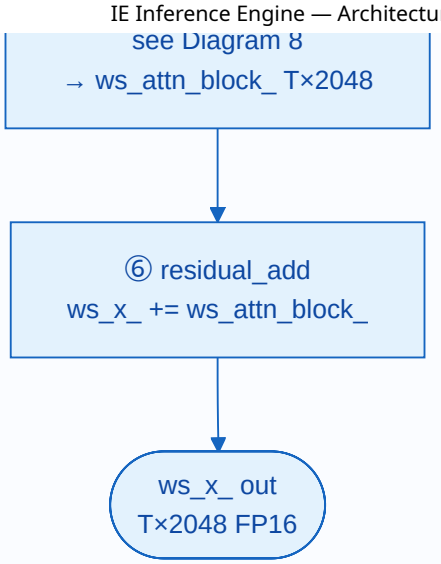
1. Top-Level Forward Pass



```
└──────────────────────────────────────────────────────────┘
      │
      ▼
┌──────────────────────────────────────────────────────────┐
│  SAMPLER  (src/ops/sampling.cpp)                         │
│  Optional: repetition_penalty                            │
│  Greedy:   sample_argmax                                 │
│  Stochastic: sample_softmax_topk_topp                    │
│    temperature → top-k → top-p → min-p → sample          │
└──────────────────────────────────────────────────────────┘
      │
      ▼
  next token ID
```

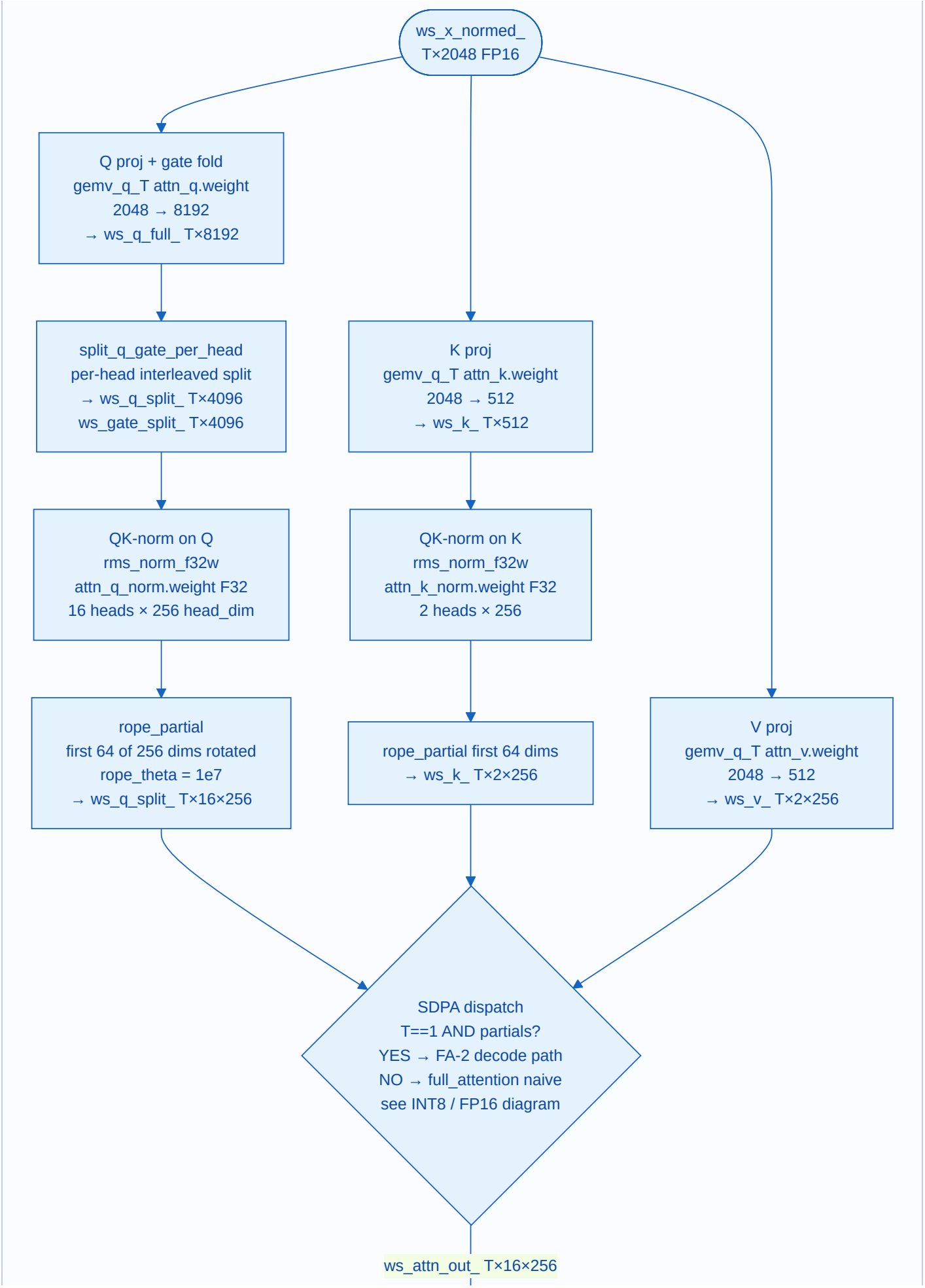
2. Per-Layer Structure (All 40 Layers)

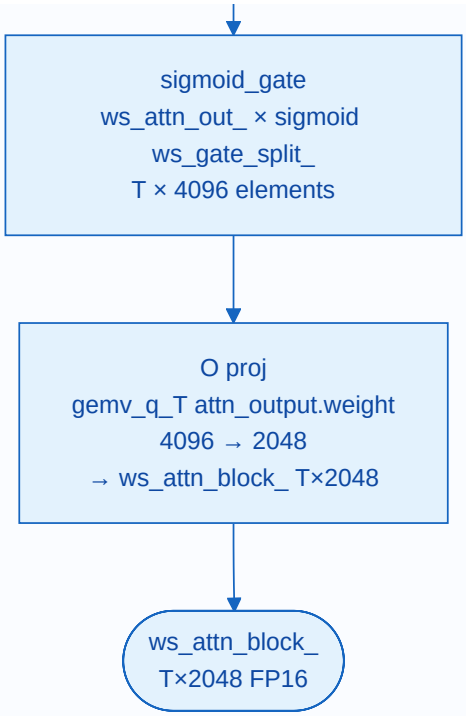

ws_x_ in
T×2048 FP16
① attn_norm
rms_norm_f32w 1+w style
weights: attn_norm.weight F32
→ ws_x_normed_
L mod 4 == 3?
YES — full-attn layers
3,7,11,15,19,23,27,31,35,39
full_attn_idx = L / 4
NO — DeltaNet layers
all other 30
dn_idx = L - L/4
② Full Attention Block
see Diagram 3
→ ws_attn_block_ T×2048
② DeltaNet Block
see Diagram 4
→ ws_attn_block_ T×2048
③ residual_add
ws_x_ += ws_attn_block_
④ post_attn_norm
rms_norm_f32w 1+w style
weights:
post_attention_norm.weight F32
→ ws_x_normed_
⑤ MoE Block
256 experts top-8


see Diagram 8
→ ws_attn_block_ T×2048
⑥ residual_add
ws_x_ += ws_attn_block_
ws_x_ out
T×2048 FP16

3. Full Attention Layer Detail (L%4 == 3)

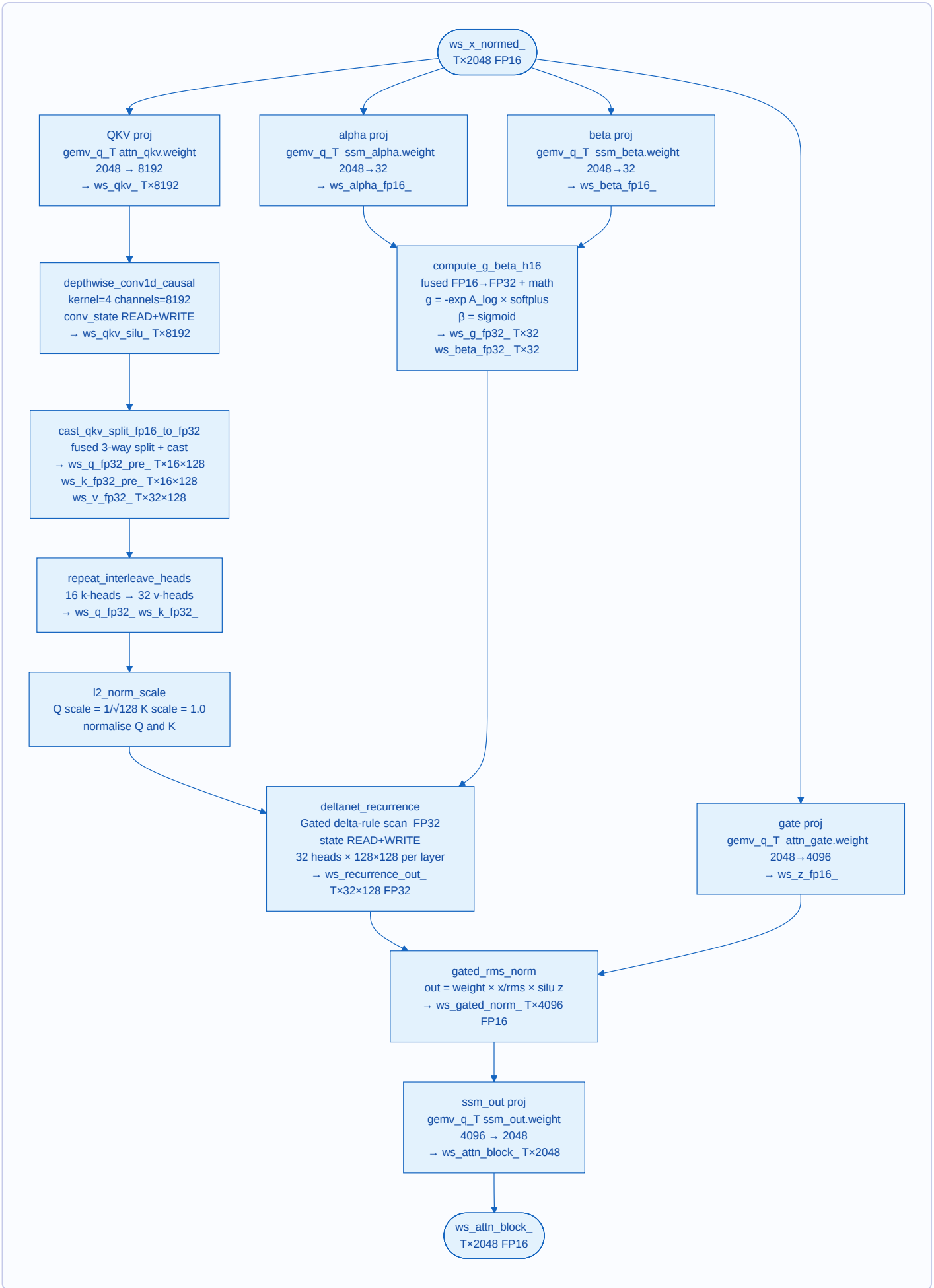
Applies to layers 3, 7, 11, 15, 19, 23, 27, 31, 35, 39. KV cache index = L / 4.


ws_x_normed_
T×2048 FP16
Q proj + gate fold
gemv_q_T attn_q.weight
2048 → 8192
→ ws_q_full_ T×8192
split_q_gate_per_head
per-head interleaved split
→ ws_q_split_ T×4096
ws_gate_split_ T×4096
K proj
gemv_q_T attn_k.weight
2048 → 512
→ ws_k_ T×512
QK-norm on Q
rms_norm_f32w
attn_q_norm.weight F32
16 heads × 256 head_dim
QK-norm on K
rms_norm_f32w
attn_k_norm.weight F32
2 heads × 256
rope_partial
first 64 of 256 dims rotated
rope_theta = 1e7
→ ws_q_split_ T×16×256
rope_partial first 64 dims
→ ws_k_ T×2×256
V proj
gemv_q_T attn_v.weight
2048 → 512
→ ws_v_ T×2×256
SDPA dispatch
T==1 AND partials?
YES → FA-2 decode path
NO → full_attention naive
see INT8 / FP16 diagram
ws_attn_out_ T×16×256


sigmoid_gate
ws_attn_out_ × sigmoid
ws_gate_split_
T × 4096 elements
O proj
gemv_q_T attn_output.weight
4096 → 2048
→ ws_attn_block_ T×2048
ws_attn_block_
T×2048 FP16

4. DeltaNet Layer Detail (L%4 != 3)

Applies to 30 of 40 layers. dn_idx = L − (L / 4). No KV cache — recurrent state only.


ws_x_normed_
T×2048 FP16
QKV proj
gemv_q_T attn_qkv.weight
2048 → 8192
→ ws_qkv_ T×8192
alpha proj
gemv_q_T ssm_alpha.weight
2048→32
→ ws_alpha_fp16_
beta proj
gemv_q_T ssm_beta.weight
2048→32
→ ws_beta_fp16_
depthwise_conv1d_causal
kernel=4 channels=8192
conv_state READ+WRITE
→ ws_qkv_silu_ T×8192
compute_g_beta_h16
fused FP16→FP32 + math
g = -exp A_log × softplus
β = sigmoid
→ ws_g_fp32_ T×32
ws_beta_fp32_ T×32
cast_qkv_split_fp16_to_fp32
fused 3-way split + cast
→ ws_q_fp32_pre_ T×16×128
ws_k_fp32_pre_ T×16×128
ws_v_fp32_ T×32×128
repeat_interleave_heads
16 k-heads → 32 v-heads
→ ws_q_fp32_ ws_k_fp32_
l2_norm_scale
Q scale = 1/√128 K scale = 1.0
normalise Q and K
deltanet_recurrence
Gated delta-rule scan FP32
state READ+WRITE
32 heads × 128×128 per layer
→ ws_recurrence_out_
T×32×128 FP32
gate proj
gemv_q_T attn_gate.weight
2048→4096
→ ws_z_fp16_
gated_rms_norm
out = weight × x/rms × silu z
→ ws_gated_norm_ T×4096
FP16
ssm_out proj
gemv_q_T ssm_out.weight
4096 → 2048
→ ws_attn_block_ T×2048
ws_attn_block_
T×2048 FP16

5. KV Cache: Every Write and Read Point

Cache Layout

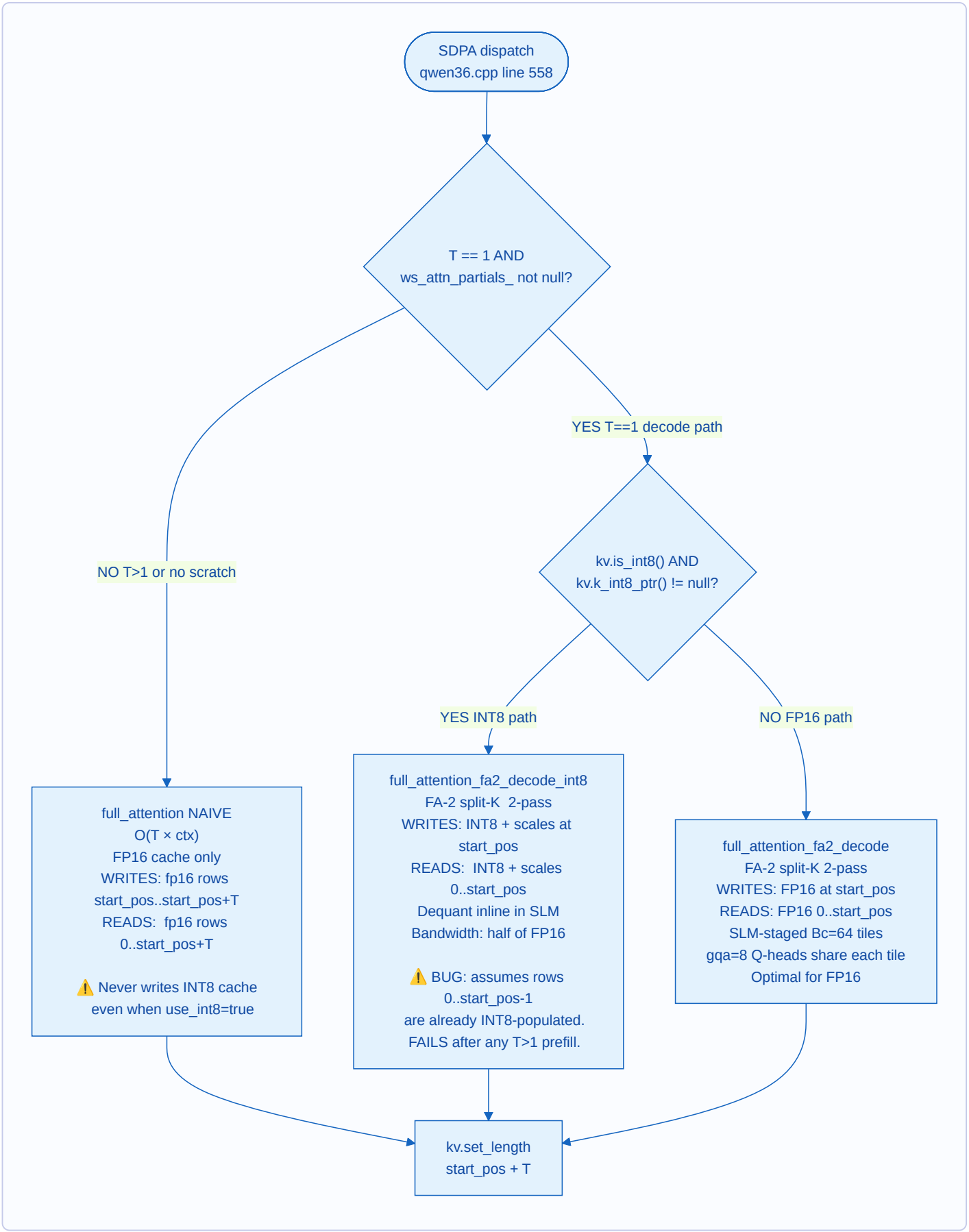
```
FP16 cache  (always allocated):
  k_cache: [L_full=10, n_kv_heads=2, max_ctx, head_dim=256]  FP16
  v_cache: [L_full=10, n_kv_heads=2, max_ctx, head_dim=256]  FP16

INT8 shadow  (only when KvCacheConfig.use_int8 == true):
  k_int8_cache: [L_full=10, n_kv_heads=2, max_ctx, head_dim=256]  INT8
  v_int8_cache: [L_full=10, n_kv_heads=2, max_ctx, head_dim=256]  INT8
  k_scales:     [L_full=10, n_kv_heads=2, max_ctx]                FP16
  v_scales:     [L_full=10, n_kv_heads=2, max_ctx]                FP16

Layer index mapping (full_attn_idx = L / 4):
  L= 3 → idx=0   L= 7 → idx=1   L=11 → idx=2   L=15 → idx=3
  L=19 → idx=4   L=23 → idx=5   L=27 → idx=6   L=31 → idx=7
  L=35 → idx=8   L=39 → idx=9
```

Kernel-by-Kernel Write / Read Map

```
full_attention  (T > 1 — prefill path)
  WRITE:  k_cache[kv, start_pos..start_pos+T-1, :]  =  k_in
          v_cache[kv, start_pos..start_pos+T-1, :]  =  v_in
          FP16 ONLY.  INT8 cache NOT touched.
  READ:   k_cache[kv, 0..start_pos+T-1, :]  (fp16, all positions)
          v_cache[kv, 0..start_pos+T-1, :]

  ⚠️  CRITICAL: If use_int8=true, these rows are never INT8-populated.
      A subsequent T=1 decode will read UNINITIALIZED INT8 values.

──────────────────────────────────────────────────────────────────────

full_attention_fa2_decode  (T == 1, is_int8 == false)
  WRITE:  k_cache[kv, start_pos, :]  =  k_in[kv, :]   (fp16)
          v_cache[kv, start_pos, :]  =  v_in[kv, :]   (fp16)
  READ:   k_cache[kv, 0..start_pos, :]   all ctx positions, fp16
          v_cache[kv, 0..start_pos, :]
  Pattern: Bc=64 tiles, CHUNKS_PER_WG=8 tiles per WG
           SLM-staged; gqa=8 Q-heads share each SLM tile

──────────────────────────────────────────────────────────────────────

full_attention_fa2_decode_int8  (T == 1, is_int8 == true)
  WRITE:  k_int8_cache[kv, start_pos*hd .. (start_pos+1)*hd]  =  quantize(k_in)
          v_int8_cache[...]  =  quantize(v_in)
```

```
          k_scales[kv, start_pos]  =  fp16(maxabs_k / 127)
          v_scales[kv, start_pos]  =  fp16(maxabs_v / 127)
          fp16 shadow: NOT written (nullptr passed in qwen36.cpp)
READ:     k_int8_cache[kv, 0..start_pos, :]   INT8, all ctx positions
          v_int8_cache[kv, 0..start_pos, :]
          k_scales[kv, 0..start_pos]
          v_scales[kv, 0..start_pos]
Dequant: fp16(int8 × scale) inline in SLM
Bandwidth: ~half of fp16 path (256 B vs 512 B per row)

⚠️  BUG: rows 0..start_pos-1 assumed INT8-populated from prior calls.
    After T>1 prefill, those rows are uninitialised (zero/garbage).
```

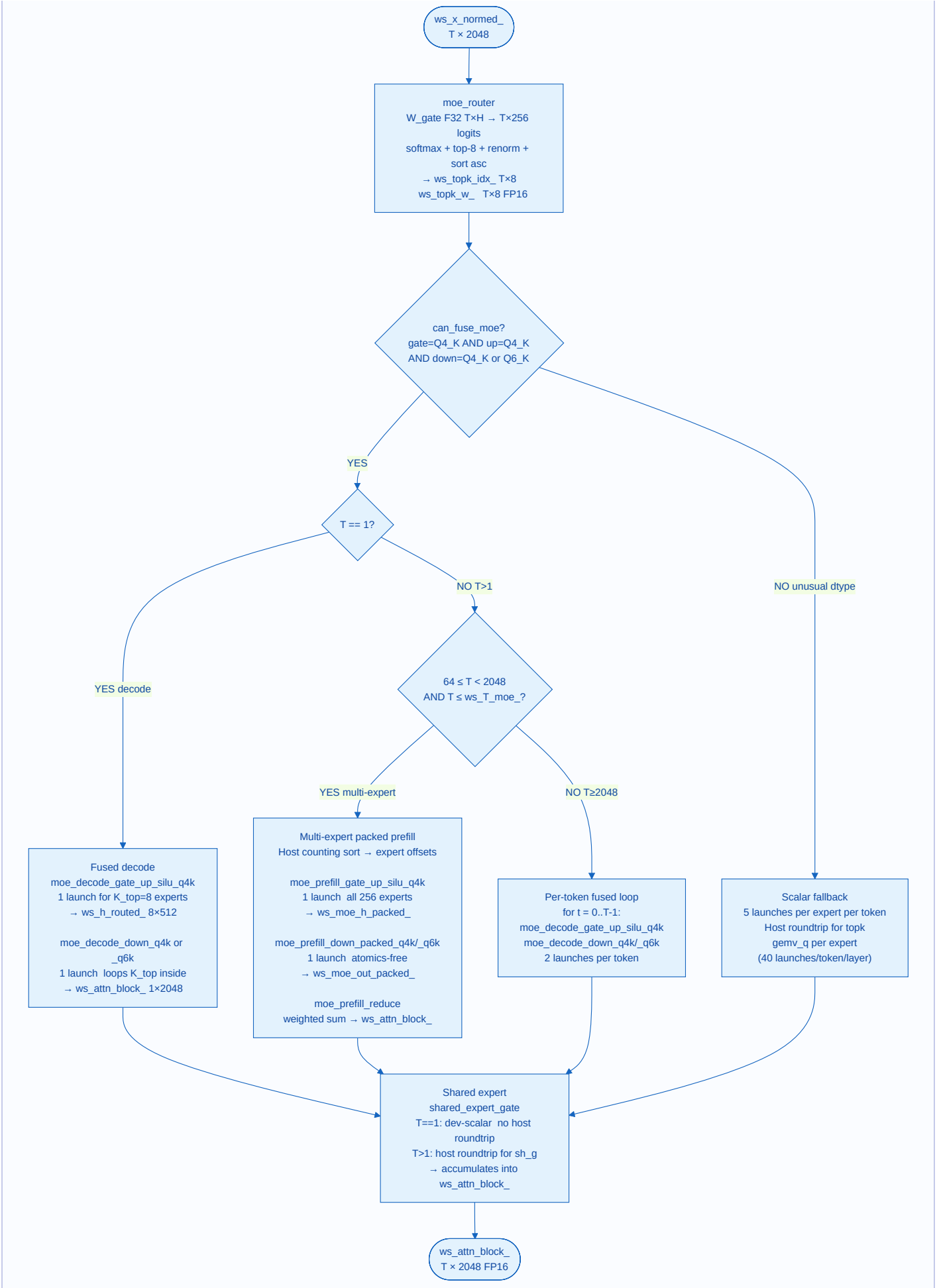
6. INT8 vs FP16 Decision Tree


SDPA dispatch
qwen36.cpp line 558
T == 1 AND
ws_attn_partials_ not null?
NO T>1 or no scratch
YES T==1 decode path
kv.is_int8() AND
kv.k_int8_ptr() != null?
YES INT8 path
NO FP16 path
full_attention NAIVE
O(T × ctx)
FP16 cache only
WRITES: fp16 rows
start_pos..start_pos+T
READS: fp16 rows
0..start_pos+T
⚠️ Never writes INT8 cache
even when use_int8=true
full_attention_fa2_decode_int8
FA-2 split-K 2-pass
WRITES: INT8 + scales at
start_pos
READS: INT8 + scales
0..start_pos
Dequant inline in SLM
Bandwidth: half of FP16
⚠️ BUG: assumes rows
0..start_pos-1
are already INT8-populated.
FAILS after any T>1 prefill.
full_attention_fa2_decode
FA-2 split-K 2-pass
WRITES: FP16 at start_pos
READS: FP16 0..start_pos
SLM-staged Bc=64 tiles
gqa=8 Q-heads share each tile
Optimal for FP16
kv.set_length
start_pos + T

7. T=1 vs T>1 Dispatch Summary

Location	T=1 path	T>1 path	Notes
gemv_q_T dispatcher	gemv_q4_K or gemv_q6_K (scalar GEMV)	gemm_q4_K or gemm_q4_K_xmx (M_TILE=8)	XMX when N%64==0 && K%256==0 and IE_NO_XMX not set
SDPA	FA-2 split-K (fp16 or INT8 variant)	Naive full_attention (O(T×ctx), fp16 only)	INT8 path only reachable at T=1
MoE routed experts	Fused decode: 2 kernels for K_top=8	64≤T<2048: multi-expert packed; T≥2048: per-token fused	See MoE diagram
Shared expert	gemv_q + scaled_add_dev_scalar (no host roundtrip)	Per-token loop + host memcpy for sh_g	T=1 avoids q.memcpy wait
INT8 KV	Available (FA-2 int8 kernel)	**NOT available — always fp16**	Root of INT8 prefill bug

8. MoE Dispatch Paths


ws_x_normed_
T × 2048
moe_router
W_gate F32 T×H → T×256
logits
softmax + top-8 + renorm +
sort asc
→ ws_topk_idx_ T×8
ws_topk_w_ T×8 FP16
can_fuse_moe?
gate=Q4_K AND up=Q4_K
AND down=Q4_K or Q6_K
YES
NO unusual dtype
T == 1?
YES decode
NO T>1
64 ≤ T < 2048
AND T ≤ ws_T_moe_?
YES multi-expert
NO T≥2048
Fused decode
moe_decode_gate_up_silu_q4k
1 launch for K_top=8 experts
→ ws_h_routed_ 8×512
moe_decode_down_q4k or
_q6k
1 launch loops K_top inside
→ ws_attn_block_ 1×2048
Multi-expert packed prefill
Host counting sort → expert offsets
moe_prefill_gate_up_silu_q4k
1 launch all 256 experts
→ ws_moe_h_packed_
moe_prefill_down_packed_q4k/_q6k
1 launch atomics-free
→ ws_moe_out_packed_
moe_prefill_reduce
weighted sum → ws_attn_block_
Per-token fused loop
for t = 0..T-1:
moe_decode_gate_up_silu_q4k
moe_decode_down_q4k/_q6k
2 launches per token
Scalar fallback
5 launches per expert per token
Host roundtrip for topk
gemv_q per expert
(40 launches/token/layer)
Shared expert
shared_expert_gate
T==1: dev-scalar no host
roundtrip
T>1: host roundtrip for sh_g
→ accumulates into
ws_attn_block_
ws_attn_block_
T × 2048 FP16

9. GEMM/GEMV Dispatch Logic

```
gemv_q_T(A[T,K], W[K,N], y[T,N], K, N, T)
│
├── T == 0  →  return (no-op)
│
├── T == 1  →  gemv_q
│               W.dt == Q4_K → gemv_q4_K
│               W.dt == Q6_K → gemv_q6_K
│
└── T > 1  AND W.dt == Q4_K:
             for m = 0, 8, 16, ...  (M_TILE = 8 chunks):
               mc = min(8, T − m)
               use_xmx = !IE_NO_XMX AND N%64==0 AND K%256==0
               use_xmx=true  → gemm_q4_K_xmx  (joint_matrix dpas)
               use_xmx=false → gemm_q4_K       (scalar, SLM-staged)

    T > 1  AND W.dt == Q6_K or other:
             scalar per-row loop: for t=0..T-1: gemv_q6_K

Notes:
  gemm_q4_K_xmx: XMX/DPAS; amortises Q4_K dequant across M_TILE rows.
  lm_head GEMV:  Always scalar gemv_q (M=1); XMX ~5% slower for M=1.
```

10. Complete Kernel Inventory

Kernel	File	Status	When Used	Notes
embedding_lookup_q4k	ops/embedding.cpp	✅ Active	Forward pass start	vocab×2048 Q4_K → T×2048 FP16
embedding_lookup_q6k	ops/embedding.cpp	✅ Active	Forward pass start	Q6_K variant
rms_norm_f32w	ops/elementwise.cpp	✅ Active	Every layer ×4 + final	Gemma3-style 1+w; FP32 weight
rope_partial	ops/elementwise.cpp	✅ Active	Full-attn Q and K	First 64 of 256 dims; theta=1e7
split_q_gate_per_head	ops/elementwise.cpp	✅ Active	Full-attn Q proj	Per-head interleaved Q/gate split
sigmoid_gate	ops/elementwise.cpp	✅ Active	Full-attn output	attn_output_gate quirk
residual_add	ops/elementwise.cpp	✅ Active	Every layer ×2 = 80/fwd	y = a + b
swiglu	ops/elementwise.cpp	✅ Active	MoE expert FFN	silu(gate)×up
scaled_add	ops/elementwise.cpp	✅ Active	MoE output (T>1)	Host scalar; y += s×x
scaled_add_dev_scalar	ops/elementwise.cpp	✅ Active	MoE shared expert T=1	Device scalar; no host roundtrip
cast_fp32_to_fp16	ops/elementwise.cpp	✅ Active	Model load	ssm_conv1d, ssm_norm
cast_qkv_split_fp16_to_fp32	ops/elementwise.cpp	✅ Active	DeltaNet	Fused 3-way split + cast; 1 launch
repeat_interleave_heads	ops/elementwise.cpp	✅ Active	DeltaNet GQA expand	16 k-heads → 32 v-heads
l2_norm_scale	ops/elementwise.cpp	✅ Active	DeltaNet Q/K norm	Q scale=1/√128; K scale=1
compute_g_beta_h16	ops/elementwise.cpp	✅ Active	DeltaNet	Fused FP16-input; saves 2 cast launches
gemv_q4_K	ops/gemv_q4k.cpp	✅ Active	T=1, Q4_K weights	~92% effective BW; decode hot path
gemv_q6_K	ops/gemv_q6k.cpp	✅ Active	T=1, Q6_K weights	
gemm_q4_K	ops/gemv_q4k.cpp	✅ Active	T>1, Q4_K, no XMX	M_TILE=8; SLM-staged activations
gemm_q4_K_xmx	ops/gemm_q4k_xmx.cpp	✅ Active	T>1, Q4_K, N%64==0	joint_matrix dpas; IE_NO_XMX=0
gemm_q6_K_xmx	ops/gemm_q4k_xmx.cpp	🔶 Declared	Not called from	Prototype present

			qwen36.cpp	
gemm_fp16	ops/gemm_fp16.cpp	✅ Active	Dense FP16 GEMM benchmark	Phase 3 baseline
gemm_q4k_esimd	ops/gemm_q4k_esimd.cpp	🧪 Experimental	Not wired into forward pass	v2.0 ESIMD research; untracked
full_attention	ops/attention.cpp	✅ Active	T>1 prefill	Naive O(T×ctx); fp16 cache only
full_attention_fa2_decode	ops/attention.cpp	✅ Active	T=1, FP16 KV	FA-2 split-K 2-pass; Bc=64; CHUNKS_PER_WG=8
full_attention_fa2_decode_int8	ops/attention.cpp	🔶 Buggy	T=1, INT8 KV flag	⚠️ Reads uninitialized rows after T>1 prefill
esimd_block2d_smoke_tile256	ops/attention.cpp	🧪 Disabled	Gated #ifdef	All variants cause device-lost on BMG G31 C0
deltanet_recurrence	ops/deltanet.cpp	✅ Active	DeltaNet layers	Gated delta-rule scan; FP32; state updated in-place
gated_rms_norm	ops/deltanet.cpp	✅ Active	DeltaNet output	FP32→FP16; out = weight × x/rms × silu(z)
depthwise_conv1d_causal	ops/conv1d.cpp	✅ Active	DeltaNet	kernel=4, 8192 channels; streaming conv_state
moe_router	ops/moe.cpp	✅ Active	Every layer	top-8 of 256; softmax + renorm + sort ascending
moe_decode_gate_up_silu_q4k	ops/moe_fused.cpp	✅ Active	T=1 fused decode	1 launch for K_top=8 experts; SLM-stages activation
moe_decode_down_q4k/q6k	ops/moe_fused.cpp	✅ Active	T=1 decode	Loops K_top inside kernel; no atomics
moe_prefill_gate_up_silu_q4k	ops/moe_fused.cpp	✅ Active	64≤T<2048	1 launch, all 256 experts packed
moe_prefill_down_packed_q4k/q6k	ops/moe_fused.cpp	✅ Active	64≤T<2048	Atomics-free; per-packed-row write
moe_prefill_reduce	ops/moe_fused.cpp	✅ Active	64≤T<2048	Weighted K_top sum per token; atomic-free
moe_prefill_gate_up_silu_q4k_xmx	ops/moe_fused.cpp	🔶 Declared	Tried; 10-15% slower	SLM dequant > dpas savings in fragmented multi-expert
moe_gather_rows	ops/moe.cpp	❌ Dead code	if(false) block	Old scatter-gather path
moe_scatter_add	ops/moe.cpp	❌ Dead code	if(false) block	Old scatter-gather path
shared_expert_gate	ops/moe.cpp	✅ Active	Every layer	sigmoid(W_shg · x) per token

sample_argmax	ops/sampling.cpp	✅ **Active**	Greedy decode	
sample_softmax_topk_topp	ops/sampling.cpp	✅ **Active**	Stochastic decode	temp → topk → topp → minp → multinomial
repetition_penalty	ops/sampling.cpp	✅ **Active**	Optional	In-place logit modification

11. Memory Layout Reference

KV Cache & DeltaNet State Memory Costs

```
KV cache (fp16, max_ctx=4096):
  per token = 10 layers × 2 kv_heads × 256 head_dim × 2 B = 10,240 B ≈ 10 KiB/tok
  4096 tokens total = 40 MiB

KV cache (INT8 shadow):
  per token ≈ 5,120 B (INT8) + 10×2×2 B (scales) ≈ 5,160 B ≈ 5 KiB/tok
  Combined fp16 + INT8: ≈ 15 KiB/tok  →  60 MiB at 4096 ctx

DeltaNet recurrent state (30 layers, CONSTANT regardless of ctx length):
  recurrent state: 30 × 32 × 128 × 128 × 4 B = 60 MiB
  conv state:      30 × 8192 × 3 × 2 B ≈ 1.5 MiB
  Total: ≈ 61.5 MiB   (never grows with context)
```

Generated from actual source code — accurate as of 2026-05-01.